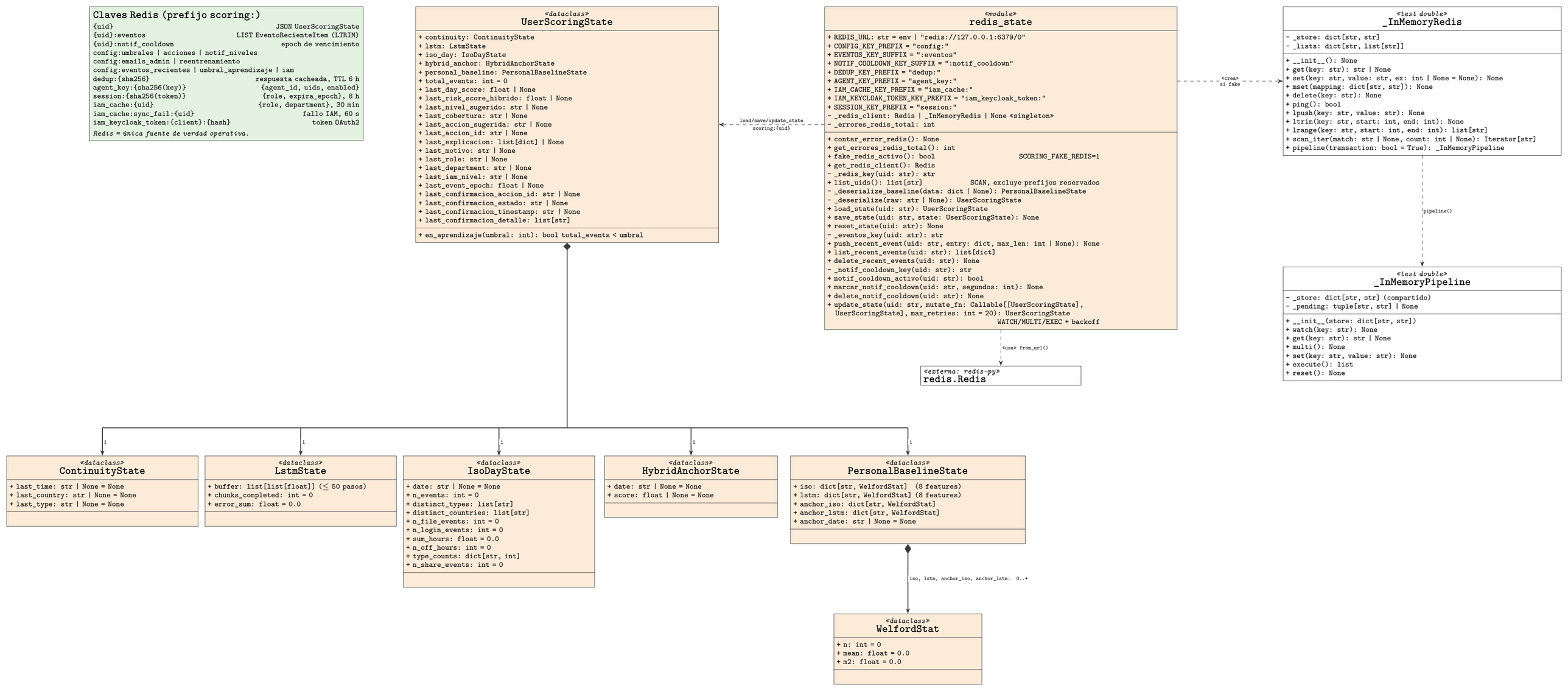

Claves Redis (prefijo scoring:)
{uid} JSON UserScoringState
{uid}:eventos LIST EventoRecienteItem (LTRIM)
{uid}:notif_cooldown epoch de vencimiento
config:umbrales | acciones | notif_niveles
config:emails_admin | reentrenamiento
config:eventos_recientes | umbral_aprendizaje | iam
dedup:{sha256} respuesta cacheada, TTL 6 h
agent_key:{sha256(key)} {agent_id, uids, enabled}
session:{sha256(token)} {role, expira_epoch}, 8 h
iam_cache:{uid} {role, department}, 30 min
iam_cache:sync_fail:{uid} fallo IAM, 60 s
iam_keycloak_token:{client}:{hash} token OAuth2
Redis = única fuente de verdad operativa.
«dataclass»
UserScoringState
+ continuity: ContinuityState
+ lstm: LstmState
+ iso_day: IsoDayState
+ hybrid_anchor: HybridAnchorState
+ personal_baseline: PersonalBaselineState
+ total_events: int = 0
+ last_day_score: float | None
+ last_risk_score_hibrido: float | None
+ last_nivel_sugerido: str | None
+ last_cobertura: str | None
+ last_accion_sugerida: str | None
+ last_accion_id: str | None
+ last_explicacion: list[dict] | None
+ last_motivo: str | None
+ last_role: str | None
+ last_department: str | None
+ last_iam_nivel: str | None
+ last_event_epoch: float | None
+ last_confirmacion_accion_id: str | None
+ last_confirmacion_estado: str | None
+ last_confirmacion_timestamp: str | None
+ last_confirmacion_detalle: list[str]
+ en_aprendizaje(umbral: int): bool total_events < umbral
load/save/update_state
scoring:{uid}
«module»
redis_state
+ REDIS_URL: str = env | "redis://127.0.0.1:6379/0"
+ CONFIG_KEY_PREFIX = "config:"
+ EVENTOS_KEY_SUFFIX = ":eventos"
+ NOTIF_COOLDOWN_KEY_SUFFIX = ":notif_cooldown"
+ DEDUP_KEY_PREFIX = "dedup:"
+ AGENT_KEY_PREFIX = "agent_key:"
+ IAM_CACHE_KEY_PREFIX = "iam_cache:"
+ IAM_KEYCLOAK_TOKEN_KEY_PREFIX = "iam_keycloak_token:"
+ SESSION_KEY_PREFIX = "session:"
- _redis_client: Redis | _InMemoryRedis | None «singleton»
- _errores_redis_total: int
+ contar_error_redis(): None
+ get_errores_redis_total(): int
+ fake_redis_activo(): bool SCORING_FAKE_REDIS=1
+ get_redis_client(): Redis
- _redis_key(uid: str): str
+ list_uids(): list[str] SCAN, excluye prefijos reservados
- _deserialize_baseline(data: dict | None): PersonalBaselineState
- _deserialize(raw: str | None): UserScoringState
+ load_state(uid: str): UserScoringState
+ save_state(uid: str, state: UserScoringState): None
+ reset_state(uid: str): None
- _eventos_key(uid: str): str
+ push_recent_event(uid: str, entry: dict, max_len: int | None): None
+ list_recent_events(uid: str): list[dict]
+ delete_recent_events(uid: str): None
- _notif_cooldown_key(uid: str): str
+ notif_cooldown_activo(uid: str): bool
+ marcar_notif_cooldown(uid: str, segundos: int): None
+ delete_notif_cooldown(uid: str): None
+ update_state(uid: str, mutate_fn: Callable[[UserScoringState], UserScoringState], max_retries: int = 20): UserScoringState
WATCH/MULTI/EXEC + backoff
«crea»
si fake
«test double»
_InMemoryRedis
- _store: dict[str, str]
- _lists: dict[str, list[str]]
+ __init__(): None
+ get(key: str): str | None
+ set(key: str, value: str, ex: int | None = None): None
+ mset(mapping: dict[str, str]): None
+ delete(key: str): None
+ ping(): bool
+ lpush(key: str, value: str): None
+ ltrim(key: str, start: int, end: int): None
+ lrange(key: str, start: int, end: int): list[str]
+ scan_iter(match: str | None, count: int | None): Iterator[str]
+ pipeline(transaction: bool = True): _InMemoryPipeline
pipeline()
«test double»
_InMemoryPipeline
- _store: dict[str, str] (compartido)
- _pending: tuple[str, str] | None
+ __init__(store: dict[str, str])
+ watch(key: str): None
+ get(key: str): str | None
+ multi(): None
+ set(key: str, value: str): None
+ execute(): list
+ reset(): None
«usa» from_url()
«externa: redis-py»
redis.Redis
1
1
1
1
1
«dataclass»
ContinuityState
+ last_time: str | None = None
+ last_country: str | None = None
+ last_type: str | None = None
«dataclass»
LstmState
+ buffer: list[list[float]] (≤ 50 pasos)
+ chunks_completed: int = 0
+ error_sum: float = 0.0
«dataclass»
IsoDayState
+ date: str | None = None
+ n_events: int = 0
+ distinct_types: list[str]
+ distinct_countries: list[str]
+ n_file_events: int = 0
+ n_login_events: int = 0
+ sum_hours: float = 0.0
+ n_off_hours: int = 0
+ type_counts: dict[str, int]
+ n_share_events: int = 0
«dataclass»
HybridAnchorState
+ date: str | None = None
+ score: float | None = None
«dataclass»
PersonalBaselineState
+ iso: dict[str, WelfordStat] (8 features)
+ lstm: dict[str, WelfordStat] (8 features)
+ anchor_iso: dict[str, WelfordStat]
+ anchor_lstm: dict[str, WelfordStat]
+ anchor_date: str | None = None
iso, lstm, anchor_iso, anchor_lstm: 0..*
«dataclass»
WelfordStat
+ n: int = 0
+ mean: float = 0.0
+ m2: float = 0.0